

Abstract Data Types

CMPT 145

Copyright Notice

©2020 Michael C. Horsch

This document is provided as is for students currently registered in CMPT 145.

All rights reserved. This document shall not be posted to any website for any purpose without the express consent of the author.

Learning Objectives

After studying this chapter, a student should be able to:

- Describe the concept of data abstraction.
- Give an example of data abstraction, and explain how it helps meet design and implementation goals.
- Define the concept of data type in terms of values and operations, and give examples.
- Define the concept of data structure in terms of values and structure, and give examples.
- Define the concept of abstract data type, in terms of what is hidden, and what is exposed, and give examples.
- Identify the data components and behaviours of multiple abstract data types.

Data Types

- A **data type** is:
 - A **set of values**
 - **Operations** that can be performed on those values.
- Some data types are provided by a language.
- New data types can be added to any language!

Data Type Examples

- Integers
 - Values: $\{\dots, -2, -1, 0, 1, 2, \dots\}$
 - In Python, the integer data type has infinite range of values.
 - In many other languages, integers have finite range!
 - Operations:
 - Addition (+)
 - Multiplication (*)
 - Modulo Division (%)
 - (and many more...)

Data Type Examples

- Booleans
 - Values: {True, False}
 - Operations:
 - OR
 - AND
 - NOT
- Some (old) languages do not have a Boolean data type, so they pretend that 0 means *False* and any other number is *True*)

Data Structures

A **data structure** is a **data type** whose values are **compound** values, **organized** in some way.

- **Compound** means **multiple values** are grouped together as part of a single collection.
- The **organization** gives the data type its **structure**.
- A data structure also has a set of operations on its values.

Data Structures Example

- Strings

- **Values:** The set of all character sequences.
 - e.g. "dog", "Open 24/7"
- **Operations:**
 - `len()`
 - Method `isdigit()`
 - Method `find()`
 - Method `replace()`

0	1	2	3	4	5	6	7	8
'O'	'p'	'e'	'n'	' '	'2'	'4'	'/'	'7'

Data Structures Examples

- Lists
 - **Values:** The set of all value sequences (any kind of value!)
 - e.g. ["behemoth", 10, 100]
 - **Operations:**
 - `len()`
 - `Method append()`
 - `Method insert()`
 - `Method reverse()`

Other Data Structure Examples

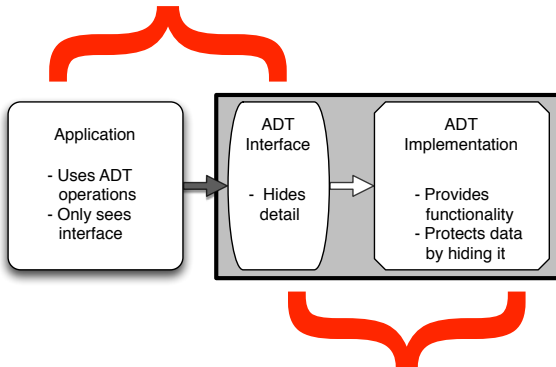
- Tuples
- Complex numbers
- Student records, employee records, Guinness World Book Records, ...
- A significant proportion of software design is choosing or designing data structures!

Abstract Data Types

- An **Abstract Data Type (ADT)** is a data type that
 1. **Hides** the data values.
Can only change values through its **operations**
 2. **Hides** the operation implementation.
Only the **interface** is known.
 3. **Hides** the details about the organizational structure.
Using the operations means **you do not need to know the details!**
- Described in terms of their:
 1. Purpose
 2. Operations
 3. Organization

How ADTs are used

Application Programmers' View



ADT Programmers' View

ADT Examples: Python Lists

- Purpose:
 - Manage linear sequences of data values.
- Implementations:
 - Data:
 - Linked Nodes
 - Fixed Length Data Arrays
 - Operations:
 - Access a value by index
 - Remove, append, insert, reverse, ...

ADT Examples: Python Dictionary

- Purpose:
 - Manage pairs of associated data items
- Implementations:
 - Data:
 - Hash Tables
 - Binary Search Trees
 - Operations:
 - Look up a value by key
 - Add/Remove a key-value pair

ADT Example: Registry

- Purpose:
 - Manage a collection of indexed Boolean values
- Implementation:
 - Data structure:
 - List of Booleans
 - Operations:
 - Create a registry
 - Set a value to True
 - Set a value to False
 - Check a given value

Demo 1

Use the Registry ADT to improve our code solution to the [Sieve of Eratosthenes](#) problem

- How does the ADT support our **design goals**?
- How does the ADT support our **implementation goals**?

Demo 2

Use the Registry ADT to improve our code solution to the **Coupon Collector** problem

- How does the ADT support our **design goals**?
- How does the ADT support our **implementation goals**?

Statistics ADT

- Purpose:
 - Computes statistics of a numerical data set
- Implementation:
 - Data Structure:
 - Record (dictionary)
 - Operations:
 - Create an empty data set
 - Add number to data set
 - Compute statistic

Demo 3

Use the Statistics ADT to enhance our code solution to the **Gambler's Ruin** problem

- How does the ADT support our **design goals**?
- How does the ADT support our **implementation goals**?